One-shot Learning for Customized Quadrotors Autotuning

Abstract— **Researchers and DIY enthusiasts are increasingly involved in building custom quadrotors. The process of building a quadrotor can be straightforward, yet one should expect various manufacturing variabilities to occur. The latter affect the performance of the robot during flight control; thus, learning how to deal with such variabilities is important. The present paper addresses this challenge in two ways. First, a constrained expectation-maximization (EM) learning algorithm is designed to capture manufacturing variabilities within a widely-used linear dynamical model for quadrotors through a single flight test. Then, the paper reports on how to autotune a system of cascade, multirate PID controllers in order to identify appropriate gains that meet certain performance certificates set by the user. The overall approach is tested on a commercially-available micro quadrotor.**

I. Introduction

The mechanical simplicity of quadrotors over other micro aerial vehicles (MAVs) has contributed to their widespread use by researchers and DIY enthusiasts alike. These vehicles offer promise in numerous applications, and can be constructed and controlled with relative easy, with little to no prior experience in system design and control. In fact, this advantage has led to a rapidly growing interest in building custom quadrotors.

However, despite the increased demand for quadrotors, difficulties in providing general controller software remain. This is because hardware configurations and performance requirements can vary depending on the user's preference (e.g., bigger/smaller quadrotors or agile/conservative ones). Existing approaches [30], [3], [16] on manual PID controller tuning or adaptive control can help to surmount this challenge. However, these approaches need user effort, rely on heuristic algorithms, or have critical tuning knobs, which should be properly adjusted by users.

To alleviate the above limitations, we propose in this paper an alternative approach to (i) capture quadrotor manufacturing variabilities using a single flight test, and (ii) optimally tune a system of cascade, multirate proportional-integral-derivative (PID) controllers. The our approach can capture hardware variabilities with a single flight test (*one-shot learning*), giving statistically meaningful identification results with sensor fusion. Once a system model is identified, controller tuning is performed by casting a PID controller into a state space model and applying optimal linear control theory to obtain the PID gains. This approach makes tuning multirate, cascade PID controllers easy by focusing on tuning each PID controller separately.

The proposed approach is divided into two parts. One part deals with learning and the other deals with tuning under the following conditions: (1) Even though hardware configurations can vary, the quadrotor model can be parameterized by physical quantities, such as the length of its wing or motor constants. (2) We assume that PID controllers in a multirate and cascade form [25] is the best controller to control quadrotors with no specific hardware configurations. (3) Users can easily obtain initial PID gains by manual tuning or transferring the gains from other similar quadrotors so that quadrotors can at least fly to collect flight data.

To learn the unknown hardware configurations, the quadrotor model is derived from Newton's second law, which allow the model to be parameterized. To learn the unknown parameters, flight data using multiple sensors (i.e., IMU and Vicon) are collected using PID controllers with initially known PID gains. Then, the unknown parameters are learned by maximizing log-likelihood of linear dynamical system (LDS) with constraints. The constrained optimization is performed by the proposed constrained expectation-maximization (EM) algorithm. The parameterized model and the constrained optimization provide prior knowledge, thereby making it possible to learn with single flight data. Also, since the EM algorithm on LDS internally uses Kalman smoothing and fuses multiple-sensor information, it helps to remove noise from observation error and thereby lowers prediction error even when learning from data from a single flight.

To tune the multirate, cascade PID controllers given the learned quadrotor model, we individually tune each PID controller by casting to a state space model. This makes it possible to apply optimal linear control theory (e.g., linear quadratic regulator (LQR)) and using LQR and PID equivalent relation [19]. Once each PID controller is optimally tuned, successive loop-closures method [12] is applied from the inner PID controller to the outer one by changing sampling period accordingly.

Our approach differs from existing approaches. To identify hardware configurations, a quadrotor model that is parameterized by physical quantities, e.g. wing length, can be assumed. Each unknown physical parameters are directly identified from physical measurements, e.g. measure the moment of inertia by using pendulum rigs [17] or from physical experiments, e.g. measure motor constants by considering the relation between force and motor inputs [25]. However, to apply this approach, users should know how to conduct the physical experiments, it is time consuming, and it requires extra devices. So this approach is not proper for identifying customized quadrotors for end users. But, we uses flight data to learn the parameterized quadrotor model. [26], [15] suggested a quadrotor identification using flight data. They use CIFER system identification toolkit developed

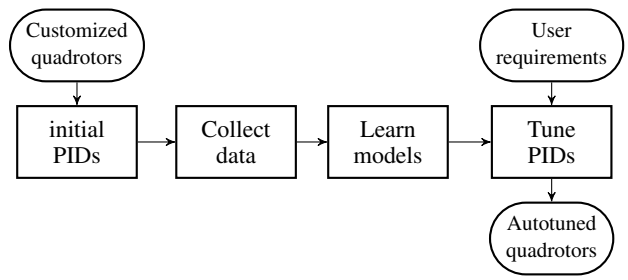

Fig. 1. Overall autotuning procedure.

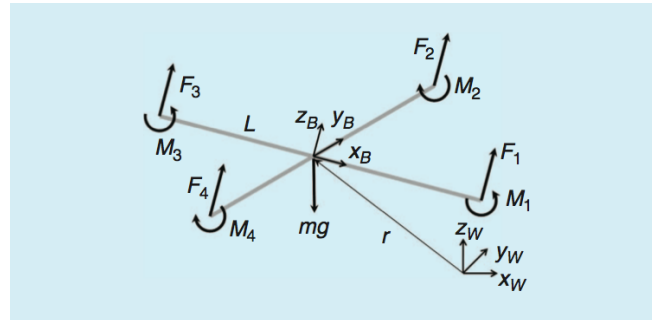

Fig. 2. Quadrotor coordinate systems [25]

by NASA Ames Research Center. However, they require user interventions, such as manually controlled quadrotor to collect clean flight data. But, our approach uses multi-sensor flight data and EM with LDS optimally smooth-out noise, it is not susceptible with noise. [13] suggested the subspace-model-based quadrotor identification. However, it considers a black box model so prone to overfitting if the amount of flight data is small. However, the proposed approach assumes the parameterized quadrotor model derived from Newton's second law, it embed only required parameters, suggesting required less flight data to achieve relatively good prediction error. Regarding PID controller tuning, single PID controller tuning has been extensively researched from heuristic to optimal methods [30], [19], [10]. If multiple PID controllers are cascaded and have different sampling periods, one doable approach includes applying the above mentioned single PID tuning methods successively from the inner to outer by changing sampling periods. However, to our understanding, there is little related work which describes overall process to tune multirate, cascade PID controllers. We review existing methods and combine them to tune multirate, cascade PID controllers.

In summary, the main contributions of this study are three fold; 1) learning and tuning the customized quadrotor automatically, 2) learning the parameterized quadrotor model using multisensor, single flight data with a constrained EM algorithm on LDS. 3) describing overall procedure to tune multirate, cascade PID controllers.

II. Problem Definition

Customized quadrotors autotuning problem is finding PID controllers gains under diverse hardware configurations and user requirements by minimizing user intervention.

In this paper, we tackle this problem by learning hardware configurations using flight data to identify diverse hardware configurations, and by tuning PID controllers given the learned model to satisfy user requirements. To minimize user intervention, we aim to learn parameterized hardware configurations with *single*, multisensored fight data. Note that we assumes that the only user requirement is stability of quadrotors and fixed response time, e.g. 1 sec. to rise. Fig. 1 describes overall procedure. Under user-selected hardware configurations, a customized quadrotor is built. Initial PID gains to collect flight data are obtained by, e.g. hand tuning. For each axes, collect flight data so that learn the corresponding quadrotor model. Given the learned model, PID gains are tuned to be satisfied user requirements.

III. Quadrotor UAV Architecture

To handle diverse hardware configurations, casting the varients into a standardized model is required. We assumes a specific quadrotor layout (Fig. 2). This seems restrictive, but most quadrotor models can be transformed into this type of layout. Also, assumes the quadrotors are controlled using PID controllers. Since controller developer cannot assumes the quadrotor hardware configuration in advance, model-based controllers cannot be used.

A. Hardware Configurations

From the quadrotor coordinate system Fig. 2 and the Newton's second law, we can correlate the motion (linear acceleration and angular acceleration) with the force and momentum of each motor as follows [25]:

$$m\begin{bmatrix}\ddot{x}\\ \ddot{y}\\ \ddot{z}\end{bmatrix} = \begin{bmatrix}0\\0\\-mg\end{bmatrix} + R\begin{bmatrix}0\\0\\\sum_i F_i\end{bmatrix}$$
$$I\begin{bmatrix}\dot{p}\\ \dot{q}\\ \dot{r}\end{bmatrix} = \begin{bmatrix}L(F_2-F_4)\\ L(F_3-F_1)\\ M_1-M_2+M_3-M_4\end{bmatrix} - \begin{bmatrix}p\\q\\r\end{bmatrix}\times I\begin{bmatrix}p\\q\\r\end{bmatrix},$$

where x,y,z is the position of a quadrotor, $\begin{bmatrix}p & q & r\end{bmatrix}^\top$ is a body rate, m is the mass of a quadrotor, g is a gravitational acceleration, R is a rotation matrix, F_i and M_i is a force and moment generated by ith motor, respectively, L is the distance from the axis of rotation of the motor to the center of the quadrotor, I is the moment of inertia.

The body rate is an orthogonal angular velocity of body. But, to control the body, Euler angle, e.g. roll/pitch/yaw, can be convenient. The following equation relates the Euler rate to the body rate.

$$\begin{bmatrix}p\\q\\r\end{bmatrix} = \begin{bmatrix}\cos\theta & 0 & -\cos\phi\sin\theta\\ 0 & 1 & \sin\phi\\ \sin\theta & 0 & \cos\phi\cos\theta\end{bmatrix}\begin{bmatrix}\dot{\phi}\\ \dot{\theta}\\ \dot{\psi}\end{bmatrix},$$

where ϕ,θ, and ψ are roll, pitch and roll, respectively.

From the second assumption of our architecture, the moment of inertia can be a diagonal matrix and the angular acceleration can be more simplified as follows:

$$\begin{bmatrix}\dot{p}\\ \dot{q}\\ \dot{r}\end{bmatrix} = \begin{bmatrix} \frac{Lk_F(\omega_2^2-\omega_4^2)}{I_{xx}} - qr\frac{I_{zz}-I_{yy}}{I_{xx}} \\ \frac{Lk_F(\omega_3^2-\omega_1^2)}{I_{yy}} - pr\frac{I_{xx}-I_{zz}}{I_{yy}} \\ \frac{k_M(\omega_1^2-\omega_2^2+\omega_3^2-\omega_4^2)}{I_{zz}} + pq\frac{I_{xx}-I_{yy}}{I_{zz}} \end{bmatrix}$$

To describe equations of motion in terms of controller inputs, the force and moment of motors can be parameterized as follows [25]:

$$F_i = k_F\omega_i^2$$
$$M_i = k_M\omega_i^2,$$

where F_i is the force of i-th motor, M_i is the moment of i-th motor, ω_i is a angular velocity of i-th motor, k_F is a motor force constant, and k_M is a motor moment constant.

From our first assumption, it is reasonable to linearize a quadrotor model at the hovering state, to assume roll and pitch angles are small so that $\begin{bmatrix}p & q & r\end{bmatrix}^\top \approx \begin{bmatrix}\dot{\phi} & \dot{\theta} & \dot{\psi}\end{bmatrix}^\top$. To obtain a parameterized linear state space model at the hovering state, we define the state, initial state, input, and initial input as follows:

$$\mathbf{x} = \begin{bmatrix}\phi & \theta & \psi & x & y & z & \dot{\phi} & \dot{\theta} & \dot{\psi} & \dot{x} & \dot{y} & \dot{z}\end{bmatrix}^\top,$$
$$\mathbf{x}_0 = \begin{bmatrix}0 & 0 & \psi_0 & x_0 & y_0 & z_0 & 0 & 0 & 0 & 0 & 0 & 0\end{bmatrix}^\top,$$
$$\mathbf{u} = \begin{bmatrix}\omega_1^2 & \omega_2^2 & \omega_3^2 & \omega_4^2\end{bmatrix}^\top,\ \mathbf{u}_0 = \begin{bmatrix}\frac{mg}{4k_F} & \frac{mg}{4k_F} & \frac{mg}{4k_F} & \frac{mg}{4k_F}\end{bmatrix}^\top.$$

Furthermore, from our second assumption, a quadrotor is symmetric along x and y axis so that $I_{xx} \approx I_{yy}$.

From the above facts, the parameterized, nonlinear state space model can be defined as follows:

$$\dot{\mathbf{x}} = f(\mathbf{x},\mathbf{u}) = \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \\ \dot{x} \\ \dot{y} \\ \dot{z} \\ \frac{Lk_F(\omega_2^2-\omega_4^2)}{I_{xx}} - \dot{\theta}\dot{\psi}\frac{I_{zz}-I_{yy}}{I_{xx}} \\ \frac{Lk_F(\omega_3^2-\omega_1^2)}{I_{yy}} - \dot{\phi}\dot{\psi}\frac{I_{xx}-I_{zz}}{I_{yy}} \\ \frac{k_M(\omega_1^2-\omega_2^2+\omega_3^2-\omega_4^2)}{I_{zz}} \\ \frac{\sum_i k_F\omega_i^2}{m}(\cos\psi\sin\theta + \cos\theta\sin\phi\sin\psi) \\ \frac{\sum_i k_F\omega_i^2}{m}(\sin\psi\sin\theta - \cos\psi\cos\theta\sin\phi) \\ -g + \frac{\sum_i k_F\omega_i^2}{m}\cos\phi\cos\theta \end{bmatrix}$$

By linearizing the nonlinear state space model at the hovering state, and discretizing the model in sampling period T, we can obtain a linear, discrete state space model as follows:

$$\delta\mathbf{x}_{t+1} = A^{(T)}\delta\mathbf{x}_t + B^{(T)}\delta\mathbf{u}_t,$$

where $A^{(T)} \approx I + T\left.\frac{\partial f}{\partial \mathbf{x}}\right|_{\mathbf{x}=\mathbf{x}_0}$, $B^{(T)} \approx T\left.\frac{\partial f}{\partial \mathbf{u}}\right|_{\mathbf{u}=\mathbf{u}_0}$, $\delta\mathbf{x}_t = \mathbf{x}_t - \mathbf{x}_0$, and $\delta\mathbf{u}_t = \mathbf{u}_t - \mathbf{u}_0$. Since thrust, roll, pitch, and yaw dynamics are isolated by design, we decompose the state space model into four models as follow:

$$\begin{bmatrix}\delta\phi_{t+1}\\ \delta\dot{\phi}_{t+1}\end{bmatrix} = \begin{bmatrix}1 & T\\ 0 & D_\phi\end{bmatrix}\begin{bmatrix}\delta\phi_t\\ \delta\dot{\phi}_t\end{bmatrix} + \begin{bmatrix}0 & 0 & 0 & 0\\ 0 & \frac{TLk_F}{I_{xx}} & 0 & \frac{-TLk_F}{I_{xx}}\end{bmatrix}\delta\mathbf{u}_t \quad (1)$$

$$\begin{bmatrix}\delta\theta_{t+1}\\ \delta\dot{\theta}_{t+1}\end{bmatrix} = \begin{bmatrix}1 & T\\ 0 & D_\theta\end{bmatrix}\begin{bmatrix}\delta\theta_t\\ \delta\dot{\theta}_t\end{bmatrix} + \begin{bmatrix}0 & 0 & 0 & 0\\ \frac{-TLk_F}{I_{yy}} & 0 & \frac{TLk_F}{I_{yy}} & 0\end{bmatrix}\delta\mathbf{u}_t \quad (2)$$

$$\begin{bmatrix}\delta\psi_{t+1}\\ \delta\dot{\psi}_{t+1}\end{bmatrix} = \begin{bmatrix}1 & T\\ 0 & D_\psi\end{bmatrix}\begin{bmatrix}\delta\psi_t\\ \delta\dot{\psi}_t\end{bmatrix} + \begin{bmatrix}0 & 0 & 0 & 0\\ \frac{Tk_M}{I_{zz}} & \frac{-Tk_M}{I_{zz}} & \frac{Tk_M}{I_{zz}} & \frac{-Tk_M}{I_{zz}}\end{bmatrix}\delta\mathbf{u}_t \quad (3)$$

$$\begin{bmatrix}\delta z_{t+1}\\ \delta\dot{z}_{t+1}\end{bmatrix} = \begin{bmatrix}1 & T\\ 0 & D_z\end{bmatrix}\begin{bmatrix}\delta z_t\\ \delta\dot{z}_t\end{bmatrix} + \begin{bmatrix}0 & 0 & 0 & 0\\ \frac{Tk_F}{m} & \frac{Tk_F}{m} & \frac{Tk_F}{m} & \frac{Tk_F}{m}\end{bmatrix}\delta\mathbf{u}_t \quad (4)$$

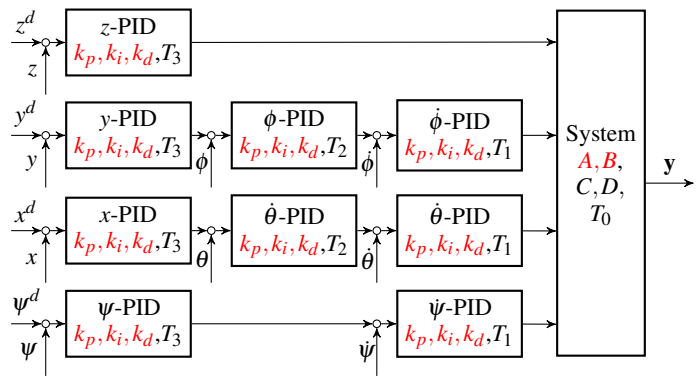

Fig. 3. PID controllers layout and quadrotor system

B. Controller Software

The controller block diagram for quadrotors is shown at Fig. 3. It consists of position controllers and attitude controllers. The position controllers include x, y, z-PID controllers, and the attitude controllers further divided into two parts: rate controllers and angle controllers. The rate controllers control roll/pitch/yaw rates and the angle controllers control roll/pitch/yaw angles. Each PID controllers has its own sampling period so that they are combined in a multirate and cascade fashion.

Once control inputs, $\mathbf{u}_{trpy} = \begin{bmatrix}u_F & u_\phi & u_\theta & u_\psi\end{bmatrix}^\top$, are generated, they are distributed to each motor, $\mathbf{u} = \begin{bmatrix}u_1 & u_2 & u_3 & u_4\end{bmatrix}^\top$, by $\mathbf{u} = M\mathbf{u}_{trpy}$, where M a motor distribution matrix. For the cross formation, the motor distribution matrix is $M_c = \begin{bmatrix}1 & 0 & -1 & 1\\ 1 & 1 & 0 & -1\\ 1 & 0 & 1 & 1\\ 1 & -1 & 0 & -1\end{bmatrix}$.

IV. Parameter One-shot Learning

To learn the parameterized quadrotor model (Eq. (1) - Eq. (4)) with single-flight data, maximizing the generalization power is important. We tackle this issue by minimizing noise from data and minimizing the number of free parameters to be learned. To minimize noise, multisensor data is fused, and observation and input error are estimated so that they can be smoothed-out when learning. Especially, inputs generated by PID controllers contain noisy due to the derivative term of a PID controller. To minimize the number

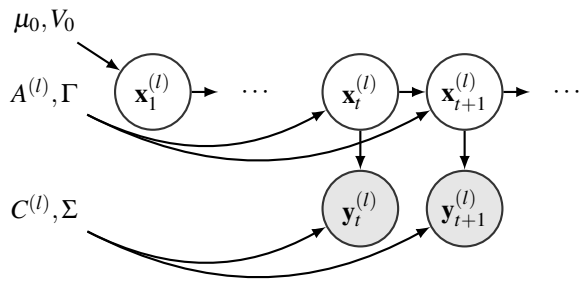

Fig. 4. The graphical model for quadrotor physical parameter learning.

of free-parameters, we constrained the model search space by using the proposed constrained EM algorithm.

A. Linear Dynamical System with Noisy Inputs

To minimize noise, we adopt a linear dynamical system (LDS), which models state and observation noises as zero-mean Gaussian, and can fuse multisensor data. To handle noise in inputs, the hidden state is augmented by inputs and the corresponding observation is augmented by noisy inputs. The final quadrotor LDS with noisy inputs is defined as follows:

$$\mathbf{x}_{t+1}^{(l)} = A^{(l)}\mathbf{x}_t^{(l)} + \varepsilon_t^{(x)} \tag{5}$$

$$\mathbf{y}_t^{(l)} = C^{(l)}\mathbf{x}_t^{(l)} + \varepsilon_t^{(y)}, \tag{6}$$

where $\mathbf{x}_t^{(l)} = \begin{bmatrix} \delta\mathbf{x}_t \\ \mathbf{v}_t \\ 1 \end{bmatrix}$, $\mathbf{y}_t^{(l)} = \begin{bmatrix} \delta\mathbf{y}_t \\ \mathbf{u}_t \\ 1 \end{bmatrix}$, $A^{(l)} = \begin{bmatrix} A & B & -B\mathbf{u}_0 \\ \mathbf{0} & & I_{5\times5} \end{bmatrix}$, $C^{(l)} = \begin{bmatrix} C & \mathbf{0} \\ \mathbf{0} & I_{5\times5} \end{bmatrix}$, $\varepsilon_t^{(x)}$ is an augmented hidden state with a Gaussian noise with zero mean and covariance Γ, $\varepsilon_t^{(y)}$ is an augmented observation with a Gaussian noise with zero mean and covariance Σ, and the initial state $\mathbf{x}_1$ is a Gaussian with mean μ_0 and covariance V_0. $\delta\mathbf{x}_t$, $\delta\mathbf{y}_t$, $\mathbf{u}_t$ and $\mathbf{u}_0$ are an original state, observation, input, and nominal input, respectively. $\mathbf{v}_t$ is a hidden state of an input. A and B are one of the decomposed state space model among Eq. 1-Eq. 4. C depends on sensor settings: we measure angular velocity, pose and position of a quadrotor, so the corresponding elements of C is set to one.

The graphical model (Fig. 4) represent dependency between random variables, by which a log-likelihood of LDS is decomposed. For simplicity the superscript (l) will be dropped in the remaining learning section.

B. Learning: Constrained EM Algorithm

Given the quadrotor LDS with noisy inputs (Eq. (5) and (6)), and a set of parameters $\theta = (\mu_0, V_0, A, \Gamma, \Sigma)$, our learning objective is to maximize the following marginal log-likelihood [14]:

$$\theta^* = \arg\max_{\theta} \ \ln \sum_{\mathbf{x}_{1:N}} p(\mathbf{y}_{1:N}, \mathbf{x}_{1:N}|\theta), \tag{7}$$

Algorithm 1 Parameter Learning using EM algorithm

procedure LEARN($\theta_0, \mathbf{y}_{1:N}, \varepsilon$)
  $\theta^{new} \leftarrow \theta_0$
  do
    $\theta^{old} \leftarrow \theta^{new}$
    $\mathbb{E}[\mathbf{x}_t] \leftarrow \hat{\mu}_t$ ▷ E-step
    $\mathbb{E}[\mathbf{x}_t\mathbf{x}_{t-1}^\top] \leftarrow \hat{V}_t J_{t-1}^\top + \hat{\mu}_t\hat{\mu}_{t-1}^\top$
    $\mathbb{E}[\mathbf{x}_t\mathbf{x}_t^\top] \leftarrow \hat{V}_t + \hat{\mu}_t\hat{\mu}_t^\top$
    $\theta^{new} \leftarrow \left(\mu_0^{new}, V_0^{new}, A^{new}, \Gamma^{new}, \Sigma^{new}\right)$ ▷ M-step
  while $\|p(\mathbf{y}_{1:N}|\theta^{new}) - p(\mathbf{y}_{1:N}|\theta^{old})\|_2 < \varepsilon$
  return θ^{new}
end procedure

where $\mathbf{x}_{1:N}$ is an abbreviation of the list of $\mathbf{x}_t$ from $t=1$ to $t=N$, $p(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t|A\mathbf{x}_{t-1}, \Gamma)$, $p(\mathbf{y}_t|\mathbf{x}_t) = \mathcal{N}(\mathbf{y}_t|C\mathbf{x}_t, \Sigma)$, $p(\mathbf{x}_1) = \mathcal{N}(\mathbf{x}_1|\mu_0, V_0)$, and N is the number of observations.

Solving Eq. (7) is intractable so EM algorithm is used to find a local maximum. The EM algorithm is an iterative algorithm which includes expectation step (E-step) and maximization step (M-step) and guarantees it always converges to local optimum. Alg. 1 describes EM algorithm. θ_0 is a set of initial parameters, ε is a sufficiently small value. $\hat{\mu}_t, \hat{V}_t$, J_{t-1} and $p(\mathbf{y}_{1:N}|\theta)$ are inferred using the forward-backward algorithm [14].

However, with single-flight data, it is prone to overfit to the data, which includes noise. To attenuate the overfitting problem, we reduce the number of free-parameters by constraining search space of A.

Let the constrained domain of A be $\mathscr{A}$, and free-parameters are the elements related to unknown variables, i.e., $D_\phi, D_\theta, D_\psi, D_z, k_F, k_M, L, m, I_{xx}, I_{yy}$, and I_{zz}. So in the maximization step of EM algorithm, new A is obtained by constrained gradient ascent by maximizing the following log-likelihood:

$$A^{new} = \arg\max_{A\in\mathscr{A}} \ -\frac{1}{2}\sum_{t=2}^{N}\sum_{i,j}\Gamma_{i,j}^{-1}\Big\{\mathbb{E}\left[x_{t,i}x_{t,j}\right] - A_j\mathbb{E}\left[x_{t,i}\mathbf{x}_{t-1}\right] - A_i\mathbb{E}\left[x_{t,j}\mathbf{x}_{t-1}\right] + A_i\mathbb{E}[\mathbf{x}_{t-1}\mathbf{x}_{t-1}^\top]A_j^\top\Big\} \tag{8}$$

where A_i is a i-th row of A. Note that unconstrained parameters such as $\mu_0^{new}, V_0^{new}, \Gamma^{new}$, and Σ^{new} have closed-from solutions [14].

Also note that, the learned quadrotor model, A and B, can have a usually different sampling period compared to inner most PID controller. Let the sampling period of the learned system be T_1 and that of the inner most controller be T_2, then the sampling period can be approximately changed so that the new quadrotor model at the sampling period T_2 is $A^{(T_2)} \approx \left(1 - \frac{T_2}{T_1}\right)I + \frac{T_2}{T_1}A^{(T_1)}$ and $B^{(T_2)} \approx \frac{T_2}{T_1}B^{(T_1)}$

C. Data Collection

For learning four different quadrotor model Eq. (1) - Eq. (4), four flight data, $\{\mathbf{y}_t, \mathbf{u}_t\}_{t=1:N}$ are collected by accelerating a quadrotor along roll, pitch, yaw, and z-axes. For each axes, a desired trajectory is generated and the corresponding position PID controller is tuned off.

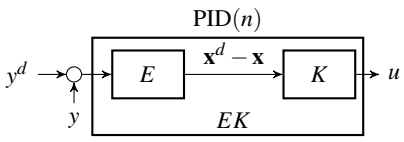

Fig. 5. A PID controller represented in a state estimator and a linear controller. n in PID(n) represents the dimension of the state. $e_t = y^d - y$, and $\mathbf{e}_t = \mathbf{x}^d - \mathbf{x}$.

To collect flight data, initial PID gains should be known in advance. We assume this can be easily achievable by heuristically adjusting PID gains.

V. Tuning Multirate, Cascade PID Controllers

PID tuning and multirate, cascade controllers are matured topics. We revisit to optially tune the multirate, cascade PID controllers.

Mathematical notations on a state space model are introduced as follows. The linear state space model S is represented in a tuple of A, B, C, D, and T such that $S = (A, B, C, D, T)$. The serial connection of two state space model $S_1 = (A_1, B_1, C_1, D_1, T)$ and $S_2 = (A_2, B_2, C_2, D_2, T)$ by setting the output of S_1 to the input of S_2 is a state space model, $S_{12} = S_1 \circ S_2 = (A_{12}, B_{12}, C_{12}, D_{12}, T)$, where $A_{12} = \begin{bmatrix} A_2 & B_2C_1 \\ \mathbf{0} & A_1 \end{bmatrix}$, $B_{12} = \begin{bmatrix} B_2D_1 \\ B_1 \end{bmatrix}$, $C_{12} = \begin{bmatrix} C_2 & D_2C_1 \end{bmatrix}$, and $D_{12} = \begin{bmatrix} D_2D_1 \end{bmatrix}$ [18]. A state space model, $S = (A, B, C, D, T)$, with a closed-loop linear controller, $u_t = -K(\mathbf{x}^d - \mathbf{x})$, can be represented in a open-loop state space model, $S^K = (A + BK, -BK, C + DK, -DK, T)$, where an input is $\mathbf{x}^d$. A state space model, $S_1 = (A, B, C, D, T_1)$ can transform to a state space model, S_2, with a sampling period, T_2 if $N = T_2/T_1$ is a positive integer. The rate converted state space model S_2 with a zero-order hold is $S^{T_2/T_1} = \left(A^N, \sum_{i=0}^{N-1} A^i B, C, D, T_2\right)$ [12].

A. A PID is a Linear Controller with a State Estimator

A PID controller is usually represented in a transfer function, but we represent it in a state space model to use optimal control theory. We assume a naive digital implementation of a PID controller so that proportional error, integral error, and derivative error are defined as $e_t = y^d - y$, $\int_0^t e_\tau = \sum_0^t T e_t$, and $\dot{e}_t = \frac{e_t - e_{t-1}}{T}$, respectively, where y^d is a desired value, y is an observation, and T is a sampling period of the PID controller.

We consider a PID controller as the combination of a state estimator and a linear controller as in Fig. 5. If a system state is $\begin{bmatrix} x_t & \int_0^t x_\tau & \dot{x}_t \end{bmatrix}^\top$, the estimator estimates the system state by assuming $y_t = x_t$. So, the state space representation of the estimator is $E^{PID} = \left(A^E, B^E, C^E, D^E\right)$, where $A^E = C^E = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ -\frac{1}{T} & 0 & 0 \end{bmatrix}$ and $B^E = D^E = \begin{bmatrix} 1 & T & \frac{1}{T} \end{bmatrix}^\top$ with an input, y_t or y_t^d, and an output, $\mathbf{x}_t = \begin{bmatrix} x_t & \int_0^t x_\tau & \dot{x}_t \end{bmatrix}^\top$ or $\mathbf{x}_t^d = \begin{bmatrix} x_t^d & \int_0^t x_\tau^d & \dot{x}_t^d \end{bmatrix}^\top$.

The linear controller part in a PID controller is represented as $u_t = K^{PID}(\mathbf{x}_t^d - \mathbf{x}_t)$, where $K^{PID} = \begin{bmatrix} k_p & k_i & k_d \end{bmatrix}$ is a PID gain matrix [19].

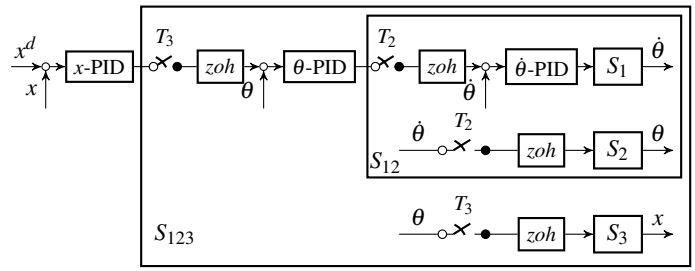

Fig. 6. x-axis PID controllers and system.

Note that the above PID controller assumes a system state is represented in $\begin{bmatrix} x_t & \int_0^t x_\tau & \dot{x}_t \end{bmatrix}^\top$. We will annotate this as PID(3) since the dimension of the system state is three. If a system state $\begin{bmatrix} x_t & \int_0^t x_\tau & \dot{x}_t \end{bmatrix}^\top$ is augmented by other states, the estimator, E^{PID}, ignores the augmented states by outputting zeros, and the first three elements of the linear controller, K^{PID}, are mapped to PID gains. If the dimension of the augmented state is n, the corresponding state space representation of a PID is annotated by PID(n).

B. Optimal Tuning of a PID controller

Once a PID controller is represented in the combination of a state estimator and a linear controller, it is straightforward to optimally tune the linear controller part using an optimal control theory (e.g., LQR). We adopt LQR to tune a PID(n) gain matrix K^{PID} (e.g., [19]).

Given a state space model, A and B, LQR can be used for tracking problem if the desired trajectory is consistent with system dynamics and the desired inputs are zero [11]. Formally, if $\mathbf{x}_{t+1}^d = A\mathbf{x}_t^d + Bu_t^d$ and $u_t^d = 0$, the error dynamics is $\mathbf{e}_{t+1} = A\mathbf{e}_t + Bv_t$, where $\mathbf{e}_t = \mathbf{x}_t^d - \mathbf{x}_t$, $v_t = u_t^d - u_t$, and $u_t = -K\mathbf{e}_t$. Since $u_t^d = 0$, the error dynamics is rewritten in $\mathbf{e}_{t+1} = A\mathbf{e}_t + B(-u_t)$. So, the optimal linear controller, K, that minimizes a quadratic cost function, $\sum_{t=0}^{\infty} \mathbf{e}_t^\top Q \mathbf{e}_t + (-u_t)^\top R(-u_t)$, is represented in a closed form and let this form as K^{LQR}, where Q and R are performance specifications. Therefore, the optimally tuned PID gain matrix K^{PID} is $-K^{LQR}$. Note that the sufficient condition for reducing a tracking problem to a regulator problem is approximately met if the desired trajectory, y_t^d, changes gradually or is constant so that the system can easily follow the trajectory. Then, the desired input, u_t can be set approximately zero and the desired trajectory, $\mathbf{x}_t^d$, generated by the state estimator, E^{PID}, approximately the state of the system so that $\mathbf{x}_{t+1}^d \approx A\mathbf{x}_t^d$.

C. Successive Loop Closure Method on Multirate and Cascade Controllers

To tune the entire multirate, cascade PID controllers, we adopt a successive loop closure method [12], [21] to tune PID controllers from inner to outer. For example, to tune x-axis PID controllers (Fig. 6), which consist of a $\dot{\theta}$-PID controller, θ-PID controller, and x-PID controller, first optimally tune the linear controller part of the inner most PID controller, K_1 with the corresponding pitch state space model S_1. The closed-loop control system is converted to the open-loop system $S_1^{K_1}$. Then, the state estimator E_1 of the inner

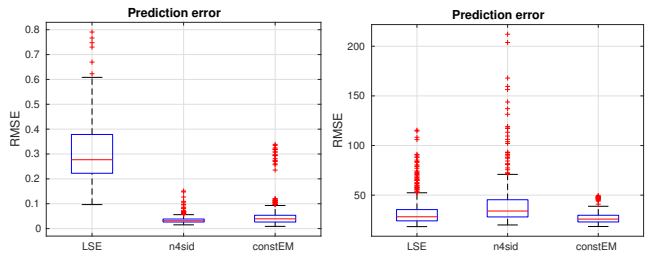

Fig. 7. z/x-axis prediction error of each learning methods. Left figure is the box plot of prediction error of z-axis learning model and right is of x-axis learning model. Both cases, the proposed constrained EM (constEM) method has sufficiently low RMSE error.

	z-axis learning		x-axis learning	
	RMSE mean	RMSE std	RMSE mean	RMSE std
LSE	0.305893	0.121026	**33.934639**	**16.752297**
n4sid	**0.035388**	**0.016509**	42.147568	25.351863
constEM	**0.054187**	**0.059315**	**27.051909**	**6.358960**

TABLE I

RMSE MEAN AND STD FOR EACH METHODS ON z/x-AXIS LEARNING. CONSTRAINED EM IS BETTER THAN OTHER METHODS IN BOTH LEARNING CASE.

most PID controller is serially concatenated to obtain new system $E_1 \circ S_1^{K_1}$. The sampling period of the new system is converted to T_2, and resulting system is $(E_1 \circ S_1^{K_1})^{T2/T1}$ so that the system can be concatenate with next system S_2. The new system S_{12} is defined by concatenating S_2 with $(E_1 \circ S_1^{K_1})^{T2/T1}$, so that $S_{12} = (E_1 \circ S_1^{K_1})^{T2/T1} \circ S_2$. Given S_{12} repeat the process until we obtain S_{123} then the outer most PID controller can be optimally tuned.

D. User Requirements

We assumes user requirements are specified by rise time and settle time. Then, a desired trajectory can be constructed and finding proper QR matrices can be solved by using a gradient descent method, which minimizes L_2 norm between the desired trajectory, r_t and the simulated trajectory, $x_t^{Q,R}$.

VI. EXPERIMENTS

A. Learning a quadrotor model

To evaluate the generalization power for each learning methods with single flight data, we consider the prediction error. The prediction error evaluated by leave-(N-1) cross validation: the model is learned using one flight data and evaluated on $N-1$ data. For each z-axis and x-axis learning, 20 flight data was collected so that each model will be evaluated on 380 times. Each evaluation, prediction error is measured by root-mean-square error (RMSE) of $\hat{y}_{t+1}$ and y_{t+1}, where $\hat{y}_{t+1}$ is prediction value given the previous observation y_t and input u_t and the learned model, and y_{t+1} is the original value. From the Fig. 7 and Table I, we could conclude that the propose constrained EM algorithm is proper for quadrotor model learning using single flight data. Since the algorithm find parameters in the constrained domain, it's less prone to overfitting on the training data.

	$\frac{k_F}{m}$	$\frac{Lk_F}{I_{yy}}$
Direct	6.7010×10^{-5}	4.0160×10^{-3}
LSE	$1.5795 \times 10^{-5} \pm 0.000010$	
constEM	$7.4285 \times 10^{-5} \pm 0.000011$	

TABLE II

ESTIMATED $\frac{k_F}{m}$ FROM z AXIS LEARNING AND ESTIMATED $\frac{Lk_F}{I_{yy}}$ FROM x AXIS LEARNING FOR EACH METHOD. EACH PARAMETERS ARE DIRECTLY MEASURED USING PHYSICAL EXPERIMENTS AND ESTIMATED BY EACH LEARNING ALGORITHM. THE PARAMETERS OF THE PROPOSED CONSTRAINED EM ALGORITHM ARE CLOSED TO THE DIRECT MEASUREMENT, SUGGESTING THE LEARNING METHOD IS RELIABLE.

	x-PID			θ-PID			$\dot{\theta}$-PID		
	k_p	k_i	k_d	k_p	k_i	k_d	k_p	k_i	k_d
init	50.0	2.0	40.0	6.0	2.5	0.0	70.0	0.0	0.0
tuned	10.6	1.2	6.3	13.2	3.2	0.9	44.8	0.0	0.0

Fig. 8. PID controller gains for x-axis. Three PID controllers are involved to control x position. The scale of initial PID gains and tuned PID gains are similar, suggesting tuned gains are reasonable.

	z-PID		
	k_p	k_i	k_d
init	2000.0	2000.0	9500.0
tuned	19195.1179	8197.8518	6465.2735

TABLE III

PID CONTROLLER GAINS FOR Z-AXIS. ONE PID CONTROLLER IS INVOLVED TO CONTROL Z POSITION. THE SCALE OF INITIAL PID GAINS AND TUNED PID GAINS ARE SIMILAR, SUGGESTING THE TUNED GAINS ARE REASONABLE.

Especially, in x-axis learning, IMU and Vicon data are used together and IMU is noisy. In this case n4sid method tends to get high RMSE error with many outliers, which suggests it also learns noise from IMU.

From the learned model, physical parameters of quadrotors can be extracted, which can be used to evaluate each learning methods. Table II shows the $\frac{k_F}{m}$, which is related to z-axis model and $\frac{Lk_F}{I_{yy}}$, which is related to x-axis model. Each physical parameters are directly obtained by physical experiments, e.g. [17]. The estimated parameters by constrained EM algorithm is quite close to the parameters from physical experiments, suggesting the proposed algorithm is reliable.

B. Tuning PID Controllers

Once the quadrotor model is learned, PID controller tuning is performed using the suggested approach. The Table 8 and III shows the PID gains of initial one used to collect flight data and the tuned gains. The scale of initial and tuned gains are similar, suggesting the tuned PID gains are reasonable.

The Fig. 9 shows the actual trajectories for each settings. The right most figure is the trajectories with turned PID gains based on the learned model using the proposed constrained EM algorithm. This learns also aerodynamics. However, the middle trajectories didn't consider the aerodynamics so that the quadrotor cannot achieve its goal and tends to be

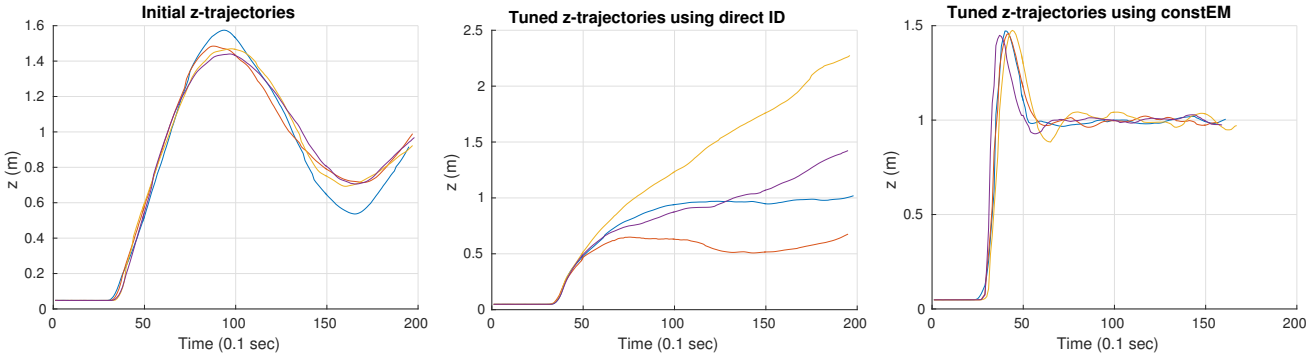

Fig. 9. z trajectories when performs "go-to-1m" with different settings. Left most one is trajectories using initial PID gains. The middle is trajectories with tuned PID gains based on directly measured quadrotor moodel. The right most one is trajectories with tuned PID gains based on the learned model using constrained EM. The middle suggests aerodynamics cannot be negligible.

diverged. This suggests learning from flight data is required to tune PID gains which satisfies user requirements.

VII. Conclusion

Autotuning method for customized quadrotors is proposed. The method divides into two parts: learning and tuning. To learn unknown hardware configurations of the quadrotors, parameterized quadrotor model is assumed. The parameters are learned using the proposed constrained EM algorithm with multi-sensor, single flight data. Experiment results show the proposed learning method has the lowest prediction error on average, suggesting the algorithm generalizes well the actual quadrotor model with limited flight data. To tune multirate, cascade PID controllers, we describes overall procedure by applying conventional optimal PID controller tuning successively from inner loop to outer by changing sampling period accordingly. The proposed autotuning method only validated on z and x-axis learning and tuning.

As the future works, y and yaw-axis autotuning should be performed. Also, the proposed autotuning need to be validated on diverse hardware configurations and user requirements. The gap between the learned model and the actual system is not negligible. To minimize the gap, hardware-in-the-loop approach is also considered.

References

[1] Arducoptor (http://copter.ardupilot.com/).

[2] Crazyflie 2.0 (https://www.bitcraze.io/crazyflie-2/).

[3] http://copter.ardupilot.com/wiki/autotune/.

[4] http://diydrones.com/.

[5] https://developer.dji.com/.

[6] https://pixhawk.org/.

[7] http://www.rcgroups.com/.

[8] P. Abbeel, A. Coates, M. Quigley, and A. Y. Ng. An application of reinforcement learning to aerobatic helicopter flight. *Advances in neural information processing systems*, 19:1, 2007.

[9] P. Abbeel and A. Y. Ng. Exploration and apprenticeship learning in reinforcement learning. In *Proceedings of the 22nd international conference on Machine learning*, pages 1–8. ACM, 2005.

[10] L. Argentim, W. Rezende, P. Santos, and R. Aguiar. Pid, lqr and lqr-pid on a quadcopter platform. In *International Conference on Informatics, Electronics & Vision*, 2013.

[11] M. Athans and P. L. Falb. *Optimal control: an introduction to the theory and its applications*. Courier Corporation, 1966.

[12] M. C. Berg, N. Amit, and J. D. Powell. Multirate digital control system design. *Automatic Control, IEEE Transactions on*, 33(12):1139–1150, 1988.

[13] M. Bergamasco and M. Lovera. Identification of linear models for the dynamics of a hovering quadrotor. *Control Systems Technology, IEEE Transactions on*, 22(5):1696–1707, 2014.

[14] C. Bishop. *Pattern recognition and machine learning*, volume 4. Springer New York, 2006.

[15] G. Cai, H. Al Mehairi, H. Al-Hosani, J. Dias, and L. Seneviratne. Frequency-domain flight dynamics model identification of mavs-miniature quad-rotor aerial vehicles. In *Intelligent Robots and Systems (IROS 2014), 2014 IEEE/RSJ International Conference on*, pages 3376–3381. IEEE, 2014.

[16] G. Chowdhary, T. Wu, M. Cutler, N. K. Ure, and J. How. Experimental results of concurrent learning adaptive controllers. In *AIAA Guidance, Navigation, and Control Conference (GNC),(Minneapolis, MN), AIAA*, 2012.

[17] L. Derafa, T. Madani, and A. Benallegue. Dynamic modelling and experimental identification of four rotors helicopter parameters. In *2006 IEEE International Conference on Industrial Technology*, 2006.

[18] A. Gattami and A. Rantzer. Linear quadratic performance criteria for cascade control. In *Decision and Control, 2005 and 2005 European Control Conference. CDC-ECC'05. 44th IEEE Conference on*, pages 3632–3637. IEEE, 2005.

[19] J.-B. He, Q.-G. Wang, and T.-H. Lee. Pi/pid controller tuning via lqr approach. In *Proceedings of the IEEE Conference on Decision and Control (CDC)*, volume 1, pages 1177–1182. IEEE, 1998.

[20] G. S. Hornby, S. Takamura, J. Yokono, O. Hanagata, T. Yamamoto, and M. Fujita. Evolving robust gaits with aibo. In *Robotics and Automation, 2000. Proceedings. ICRA'00. IEEE International Conference on*, volume 3, pages 3040–3045. IEEE, 2000.

[21] J. P. How, E. Frazzoli, and G. V. Chowdhary. Linear flight control techniques for unmanned aerial vehicles. In *Handbook of Unmanned Aerial Vehicles*, pages 529–576. Springer, 2015.

[22] N. Kohl and P. Stone. Policy gradient reinforcement learning for fast quadrupedal locomotion. In *Robotics and Automation, 2004. Proceedings. ICRA'04. 2004 IEEE International Conference on*, volume 3, pages 2619–2624. IEEE, 2004.

[23] S. Koos, J.-B. Mouret, and S. Doncieux. The transferability approach: Crossing the reality gap in evolutionary robotics. *Evolutionary Computation, IEEE Transactions on*, 17(1):122–145, 2013.

[24] Y. Lee, S. Park, and M. Lee. Pid controller tuning to obtain desired closed loop responses for cascade control systems. *Industrial & engineering chemistry research*, 37(5):1859–1865, 1998.

[25] N. Michael, D. Mellinger, Q. Lindsey, and V. Kumar. The grasp multiple micro-uav test bed. *IEEE Robotics & Automation Magazine*, 17(3):56–65, 2010.

[26] D. S. Miller. Open loop system identificaiton of a micro quadrotor helicopter from closed loop data. Master's thesis, University of Maryland, College Park, 2011.

[27] I. Sa and P. Corke. System identification, estimation and control for a cost effective open-source quadcopter. In *Robotics and Automation (ICRA), 2012 IEEE International Conference on*, pages 2202–2209. IEEE, 2012.

[28] S. Skogestad. Simple analytic rules for model reduction and pid controller tuning. *Journal of process control*, 13(4):291–309, 2003.

[29] P. Van Overschee and B. De Moor. *Subspace identification for linear systems: TheoryImplementationApplications*. Springer Science & Business Media, 2012.

[30] J. Ziegler and N. B. Nichols. Optimum settings for automatic controllers. *Transactions of the ASME*, 1942.

VIII. Memo

- Ardupilot motor distribution matrix

$$
\begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix}
=
\begin{bmatrix}
1 & \cos\left(\frac{3\pi}{4}\right) & \cos\left(\frac{\pi}{4}\right) & 1 \\
1 & \cos\left(\frac{-\pi}{4}\right) & \cos\left(\frac{-3\pi}{4}\right) & 1 \\
1 & \cos\left(\frac{\pi}{4}\right) & \cos\left(\frac{-\pi}{4}\right) & -1 \\
1 & \cos\left(\frac{5\pi}{4}\right) & \cos\left(\frac{3\pi}{4}\right) & -1
\end{bmatrix}
\begin{bmatrix} t \\ r \\ p \\ y \end{bmatrix}
$$

$$
=
\begin{bmatrix}
1 & -\cos\left(\frac{\pi}{4}\right) & \cos\left(\frac{\pi}{4}\right) & 1 \\
1 & \cos\left(\frac{\pi}{4}\right) & -\cos\left(\frac{\pi}{4}\right) & 1 \\
1 & \cos\left(\frac{\pi}{4}\right) & \cos\left(\frac{\pi}{4}\right) & -1 \\
1 & -\cos\left(\frac{\pi}{4}\right) & -\cos\left(\frac{\pi}{4}\right) & -1
\end{bmatrix}
\begin{bmatrix} t \\ r \\ p \\ y \end{bmatrix}
$$